

Análise de Código Java

LevelOrder — Travessia em Largura de Árvore Binária

Linguagem	Algoritmo	Complexidade	Nível
Java	BFS / Fila	$O(n)$ tempo e espaço	Intermediário

■ OBJETIVO GERAL

O código implementa a travessia em largura (*Level-Order Traversal*) de uma árvore binária. O objetivo é retornar os valores dos nós organizados por nível, onde cada sublista representa todos os nós de uma mesma profundidade da árvore.

■ LÓGICA E ALGORITMO (BFS)

O algoritmo utiliza **Busca em Largura (BFS)** com uma fila (*Queue*). A raiz é inserida primeiro; em cada iteração do *while*, captura-se o tamanho atual da fila (que equivale ao número de nós do nível corrente) e processa-se exatamente esse número de nós, adicionando-os a uma lista parcial. Os filhos de cada nó processado são enfileirados para o próximo nível. Ao final de cada iteração, a lista do nível é anexada ao resultado final.

■ CONCEITOS E RECURSOS JAVA ENVOLVIDOS

Classe interna estática (*static nested class*) `TreeNode` para modelar os nós. **Generics** em `List<List<Integer>>` e `Queue<TreeNode>` garantem segurança de tipos em tempo de compilação. **Interface Queue** implementada por `LinkedList`, adequada para operações FIFO eficientes com `add()` e `poll()`. Uso de **polimorfismo** ao declarar a variável como `Queue` mas instanciar como `LinkedList`.

■ COMPLEXIDADE DE TEMPO E ESPAÇO

A complexidade de **tempo** é $O(n)$, pois cada nó da árvore é visitado exatamente uma vez. A complexidade de **espaço** também é $O(n)$: no pior caso (árvore perfeitamente balanceada), o último nível contém $n/2$ nós armazenados simultaneamente na fila, além da lista de resultados que também armazena todos os n valores.

■ DECISÕES DE DESIGN E LIMITAÇÕES

A captura do tamanho da fila no início de cada iteração (`for (int i = q.size(); i > 0; i--)`) é uma decisão elegante para delimitar os níveis sem estruturas auxiliares extras. Como limitação, a classe `TreeNode` é construída manualmente e não reutilizável fora do contexto; em produção, seria preferível uma hierarquia de classes separada. A solução também não trata árvores com ciclos, pois assume estrutura de árvore estrita.

Trecho Central do Algoritmo

```
Queue<TreeNode> q = new LinkedList<>();
q.add(root);
while (!q.isEmpty()) {
    List<Integer> level = new ArrayList<>();
    for (int i = q.size(); i > 0; i--) { // delimita o nível atual
        TreeNode n = q.poll();
        level.add(n.val);
        if (n.left != null) q.add(n.left);
        if (n.right != null) q.add(n.right);
    }
    res.add(level);
}
```

Saída Esperada para o Exemplo do main()

Nível	Nós	Saída
0 (raiz)	3	[3]
1	9, 20	[9, 20]
2	15, 7	[15, 7]

Resultado final: [[3], [9, 20], [15, 7]]